

Task-02: FastSpace E-Commerce Checkout & Payment System

Full Shopping & Payment Flow | Techloom.ai Intern Assessment

1. Technical Specifications

Component	Technology	Notes
Backend	Node.js + Express.js	REST API, all business logic
Frontend	React.js (Vite)	SPA with Tailwind CSS, named FastSpace Online Store
Database	MySQL	Relational DB with transactions and row locking
Deployment - Frontend	Vercel	Auto-deploy from GitHub
Deployment - Backend	Railway	Node.js service with env variables
Deployment - Database	Railway MySQL Plugin	Attached to backend service
Version Control	GitHub	Public repository, single repo with /task-02 folder

2. Architecture Overview

Single backend architecture using Node.js Express. The React.js frontend (branded as FastSpace Online Store) communicates with the backend via REST API. MySQL handles all persistent data with transactions for stock safety.

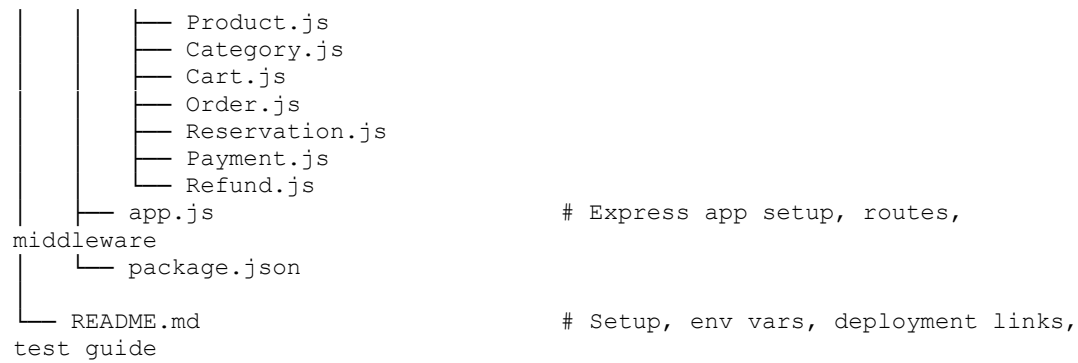
Concurrency Strategy: Stock deductions at checkout use MySQL transactions with SELECT ... FOR UPDATE row-level locking to prevent overselling when multiple customers try to buy the last available item simultaneously.

Reservation Timer: A background cron job (node-cron) checks every 60 seconds for reservations where expires_at < NOW(). Expired reservations release stock back to available inventory and the order moves to Expired status.

3. Folder Structure

```
task-02/
├── Frontend/
│   ├── public/
│   └── src/
│       ├── pages/
│       │   ├── Home.jsx           # Discovery page - product listing,
│       │   ├── search, filter, pagination
│       │   └── ItemDetails.jsx     # Product detail view - in-stock and
│       │                                   NO STOCK states
│       ├── Cart.jsx               # Shopping cart + user details +
│       │   reservation timer + payment
│       ├── PaymentSuccess.jsx     # Success screen with order summary +
│       │   auto-redirect timer
│       ├── PaymentFailed.jsx     # Failed screen - stock released,
│       │   option to cancel
│       └── PaymentTimeout.jsx     # Timeout screen - session expired,
```

stock released	Cancel.jsx	# Order cancellation with refund
method selection	OrderHistory.jsx	# User order history with status
badges		
	components/	
	Navbar.jsx	# Top nav with Category dropdown,
search bar, cart icon	CategorySidebar.jsx	# Left sidebar: Category list + Filter
results section	ProductCard.jsx	# Product image card with item name
and Add to Cart button	SimilarItems.jsx	# Right panel on ItemDetails - similar
product list	CartItem.jsx	# Single row in shopping cart (item
name + price)	ReservationTimer.jsx	# Countdown: "Your items are reserved
for X:XX"	PaymentForm.jsx	# Card form: Name, Number, Expiry,
CVV, USE THIS CARD	OrderStatusBadge.jsx	# Coloured badge for order status in
history		
	Sidebar.jsx	# General navigation sidebar
	Footer.jsx	# Footer component
	services/	
search, filter, details	productService.js	# API calls for product listing,
	cartService.js	# API calls for cart CRUD operations
	orderService.js	# API calls for order creation,
history, cancellation	reservationService.js	# API calls for stock reservation and
release	paymentService.js	# API calls for mock payment
processing	refundService.js	# API calls for refund simulation
	styles/	
	globals.css	
	App.jsx	
	main.jsx	
	index.css	
	index.html	
	vite.config.js	
	tailwind.config.js	
	package.json	
	Backend/	
	config/	
	db.js	# MySQL connection pool config
	controllers/	
single product	productController.js	# Product listing, search, filter,
	cartController.js	# Add, update, remove, clear cart
	orderController.js	# Create order, order history, cancel
	reservationController.js	# Reserve stock, release, expiry cron
	paymentController.js	# Mock payment: success / failed /
timeout	refundController.js	# Refund simulation on cancellation
	routes/	
	productRoutes.js	
	cartRoutes.js	
	orderRoutes.js	
	reservationRoutes.js	
	paymentRoutes.js	
	refundRoutes.js	
	middleware/	
	errorHandler.js	# Global error handling middleware
	validateRequest.js	# Input validation middleware
	models/	



4. UI Pages & Mockup Flow

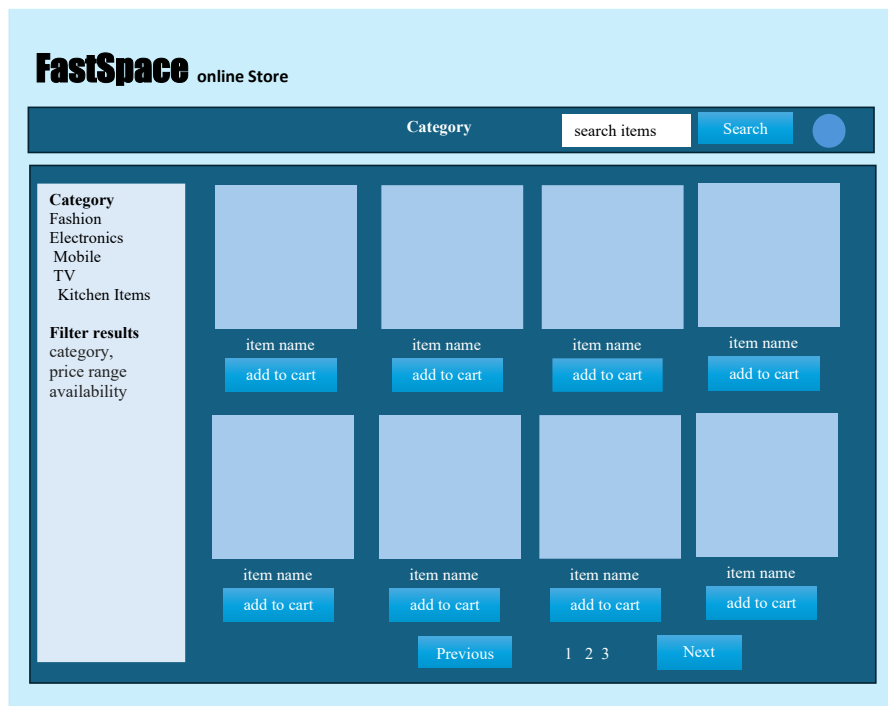


Figure 1: Discovery page

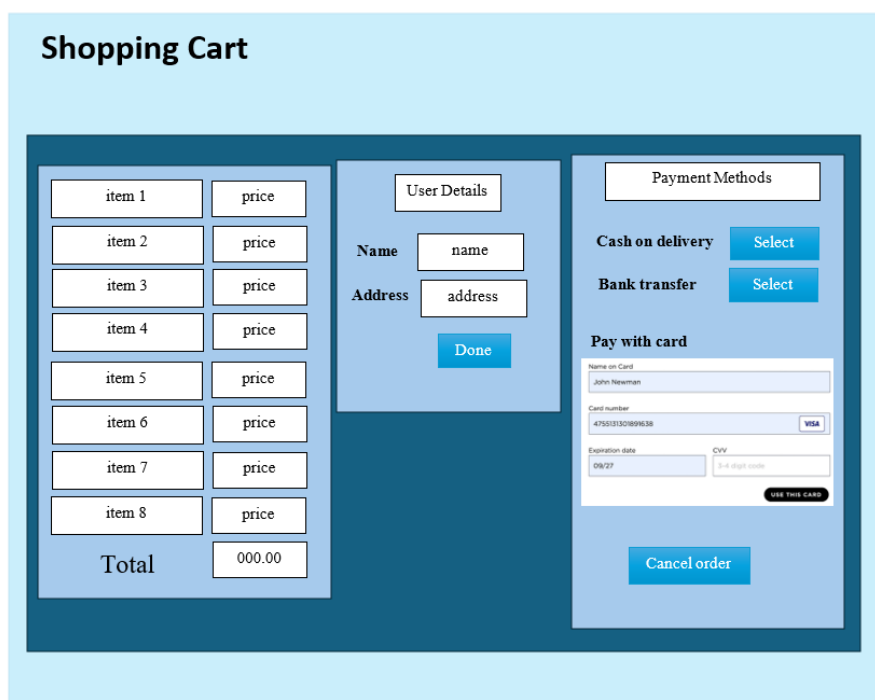


Figure 2: Cart &Payment pages

4.1 Home / Discovery Page (Home.jsx) — Figure 1

Brand: FastSpace Online Store logo top-left.

TOP NAV BAR: Category dropdown button (centre), Search items input field, Search button, Cart icon button (top-right).

MAIN LAYOUT - Two-column:

LEFT SIDEBAR: Category heading with clickable list: Fashion, Electronics, Mobile, TV, Kitchen Items. Filter Results section below with: category filter, price range filter, availability filter (in stock / out of stock).

RIGHT PRODUCT GRID: 4-column grid of product cards. Each card shows: product image (image placeholder), item name label below image, 'add to cart' button. Two rows visible with 8 products total. Pagination bar at bottom: Previous | 1 2 3 | Next.

4.2 Item Details Page (ItemDetails.jsx) — Figures 6 & 7

Navigated to when user clicks on a product card from the Discovery page.

IN-STOCK STATE (Figure 6):

LEFT SECTION: Large product image (top). Thumbnail images row below for multiple product views. Product Name heading. Quantity selector: minus button | number | plus button. Buy Now button and Add to Cart button side by side. Product Description section with full text description.

RIGHT PANEL - Similar Items: List of 4 similar products. Each has: product name label (orange background), View button, Add to Cart button.

NO STOCK STATE (Figure 7):

Same layout as in-stock but: Product image has 'NO STOCK' red watermark overlay. Quantity shows 0. Buy Now and Add to Cart buttons are greyed out and disabled. Similar Items panel still active so user can browse alternatives.

4.3 Shopping Cart & Payment (Cart.jsx) — Figure 2

Reached after user clicks 'add to cart' and proceeds to checkout.

LEFT PANEL - Cart Items: Scrollable list of items with item name and price per row. Total row at the bottom showing grand total in LKR.

CENTRE PANEL: User Details box at top - Name input field, Address input field, Done button. Reservation Timer box below: red text 'Your items are reserved for' with countdown timer showing X:XX (counts down from 5:00). Timer starts when checkout page loads and stock is reserved.

RIGHT PANEL - Payment Methods: Cash on Delivery with Select button. Bank Transfer with Select button. Pay with Card section: Name on Card input, Card Number input with

VISA logo, Expiration Date input, CVV input. USE THIS CARD button to confirm card details. Cancel Order button at bottom.

4.4 Payment Success (PaymentSuccess.jsx) — Figure 3

Displayed when mock payment gateway returns success.

Shows: 'Your Payment Success' in green text. Order Summary box with bulleted item list and Total in LKR. Auto-redirect section: 'You will be redirected to home in X:XX' with countdown timer. Back to Home button.

Backend action: Order status set to Paid. Reservation record set to completed. Stock permanently deducted.

4.5 Payment Failed (PaymentFailed.jsx) — Figure 4

Displayed when mock payment gateway returns failure.

Shows: 'Your Payment Failed' in red text. Order Summary box with items and total. Message: 'Payment method failed. Your reserved stock has been released.' Two buttons: Update Payment Method (returns to Cart page) | Cancel Order (cancels and confirms stock released).

Backend action: Order status set to Failed. Reservation released immediately. Stock restored to available inventory.

4.6 Payment Timeout (PaymentTimeout.jsx)

Displayed when mock payment gateway returns a timeout (no response after set period).

Shows: 'Payment Request Timed Out' in red/orange text. Message: 'Your session has expired. Your reservation has been released and stock is available again.' Only one button: Back to Home.

Backend action: Order status set to Expired. Reservation released. Stock restored. No retry allowed - user must start a new cart session.

4.7 Order Cancellation (Cancel.jsx) — Figure 3 (previous set)

Reached from Order History page or PaymentFailed screen.

LEFT PANEL: Product image placeholder. Item Details label. Reason text input field. Select Reason dropdown button. Cancel Order confirmation button.

RIGHT PANEL - Refund Methods (shown only if order was already paid): Card option with Select button. Bank Transfer option with Select button. Proceed button to confirm refund.

Backend action: Order status set to Cancelled. If previously Paid, a refund record is created. Stock is restored.

4.8 Order History (OrderHistory.jsx)

Accessible from the user profile or nav menu.

Displays a table of all past orders for the logged-in user:

Columns: Order ID | Items | Total (LKR) | Date | Status Badge.

Status badges use distinct colours: Paid = green, Pending = yellow, Cancelled = red, Expired = grey, Failed = orange.

Each row has a 'View Details' link showing the full order with item list. Cancelled/Failed orders show a 'Request Refund' button if not already refunded.

5. Database Design

Database name: ecommerce_database

5.1 categories table

```
CREATE TABLE categories (  
    category_id INT AUTO_INCREMENT PRIMARY KEY,  
    name        VARCHAR(50) NOT NULL UNIQUE,  
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Seed data: Fashion, Electronics, Mobile, TV, Kitchen Items
```

5.2 products table

```
CREATE TABLE products (  
    product_id    INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) NOT NULL,  
    description    TEXT,  
    category_id   INT NOT NULL,  
    price         DECIMAL(10, 2) NOT NULL,  
    stock_quantity INT NOT NULL DEFAULT 0,  
    image_url      VARCHAR(255),  
    created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
    updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
    FOREIGN KEY (category_id) REFERENCES categories(category_id)  
);
```

5.3 users table

```
CREATE TABLE users (  
    user_id      INT AUTO_INCREMENT PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    email        VARCHAR(100) NOT NULL UNIQUE,  
    address      VARCHAR(255),  
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

5.4 cart_items table

```
CREATE TABLE cart_items (  
    cart_item_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id      INT NOT NULL,  
    product_id   INT NOT NULL,  
    quantity     INT NOT NULL DEFAULT 1,  
    added_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (product_id) REFERENCES products(product_id),  
    UNIQUE KEY unique_user_product (user_id, product_id)  
);
```

5.5 orders table

```
CREATE TABLE orders (  
    order_id      INT AUTO_INCREMENT PRIMARY KEY,
```

```

        user_id          INT NOT NULL,
        total_amount     DECIMAL(10, 2) NOT NULL,
        status           ENUM('Pending', 'Reserved', 'Paid', 'Cancelled',
'Expired', 'Failed') NOT NULL DEFAULT 'Pending',
        created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at       DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
        FOREIGN KEY (user_id) REFERENCES users(user_id)
    );

```

5.6 order_items table

```

CREATE TABLE order_items (
    order_item_id        INT AUTO_INCREMENT PRIMARY KEY,
    order_id             INT NOT NULL,
    product_id           INT NOT NULL,
    quantity             INT NOT NULL,
    price_at_purchase    DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

5.7 reservations table

```

CREATE TABLE reservations (
    reservation_id       INT AUTO_INCREMENT PRIMARY KEY,
    order_id             INT NOT NULL,
    product_id           INT NOT NULL,
    quantity             INT NOT NULL,
    reserved_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
    expires_at           DATETIME NOT NULL,
    status               ENUM('active', 'expired', 'completed', 'released') NOT
NULL DEFAULT 'active',
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

```

-- expires_at = reserved_at + INTERVAL 5 MINUTE
-- Cron job: every 60 seconds check for active reservations where expires_at
< NOW()
-- On expiry: set status = expired, restore product stock, set order status
= Expired

```

5.8 payments table

```

CREATE TABLE payments (
    payment_id           INT AUTO_INCREMENT PRIMARY KEY,
    order_id             INT NOT NULL UNIQUE,
    method               ENUM('cash_on_delivery', 'bank_transfer', 'card') NOT
NULL,
    status               ENUM('success', 'failed', 'timeout') NOT NULL,
    amount               DECIMAL(10, 2) NOT NULL,
    idempotency_key      VARCHAR(100) NOT NULL UNIQUE,
    created_at           DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

```

```

-- idempotency_key: UUID generated on frontend when checkout page loads
-- Prevents duplicate payment submissions (double-click or network retry)

```

5.9 refunds table

```

CREATE TABLE refunds (
    refund_id            INT AUTO_INCREMENT PRIMARY KEY,
    order_id             INT NOT NULL,
    payment_id           INT NOT NULL,
    amount               DECIMAL(10, 2) NOT NULL,
    method               ENUM('card', 'bank_transfer') NOT NULL,
    status               ENUM('pending', 'processed') NOT NULL DEFAULT 'pending',
    created_at           DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```

        FOREIGN KEY (order_id) REFERENCES orders(order_id),
        FOREIGN KEY (payment_id) REFERENCES payments(payment_id)
    );

-- Created when a Paid order is cancelled
-- Simulates refund: status set to processed immediately in mock
implementation

```

6. API Endpoints

Base URL: <http://localhost:5000/api>

6.1 Products

Method	Endpoint	Description
GET	/products	List products with optional query params: ?search=&category=&minPrice=&maxPrice=&available=true&page=&limit=
GET	/products/:id	Get single product details (name, description, price, stock, images, similar items)
POST	/products	Add new product (admin)
PUT	/products/:id	Update product details or stock (admin)
DELETE	/products/:id	Delete product (admin)

6.2 Categories

Method	Endpoint	Description
GET	/categories	List all categories for sidebar filter

6.3 Cart

Method	Endpoint	Description
GET	/cart	Get all cart items for the current user
POST	/cart	Add item to cart { product_id, quantity }
PUT	/cart/:id	Update quantity of a cart item
DELETE	/cart/:id	Remove a single item from cart
DELETE	/cart	Clear entire cart after order creation

6.4 Orders

Method	Endpoint	Description
POST	/orders	Create order from cart items
GET	/orders	Get order history for current user with status
GET	/orders/:id	Get full order details with items and payment info

PATCH	/orders/:id/cancel	Cancel an order and restore stock. Triggers refund if previously paid
-------	--------------------	---

6.5 Reservations

Method	Endpoint	Description
POST	/reservations	Reserve all cart items for 5 minutes when checkout page loads
DELETE	/reservations/:id	Manually release a reservation

6.6 Payments

Method	Endpoint	Description
POST	/payments/process	Process mock payment. Randomly returns success, failed, or timeout outcome

Request body:

```
{
  "order_id": 1,
  "method": "cash_on_delivery" | "bank_transfer" | "card",
  "amount": 5000.00,
  "idempotency_key": "uuid-from-frontend"
}
```

6.7 Refunds

Method	Endpoint	Description
POST	/refunds	Create a refund for a cancelled paid order { order_id, method }
GET	/refunds/:order_id	Get refund status for a specific order

7. Key Technical Notes

1. SEARCH & FILTER: The GET /products endpoint uses SQL WHERE clauses with LIKE for search and equality checks for category, BETWEEN for price range, and stock_quantity > 0 for availability filter. Results are paginated using LIMIT and OFFSET.
2. CONCURRENCY: All stock deductions during reservation use MySQL transactions with SELECT ... FOR UPDATE on the products row. This prevents two users from reserving the last item simultaneously.
3. RESERVATION EXPIRY: node-cron runs every 60 seconds. Expired active reservations trigger stock restoration and order status update to Expired.

4. DUPLICATE PAYMENT: The idempotency_key (UUID) is unique in the payments table. If a duplicate request arrives with the same key, the backend returns the existing result without processing again.

5. REFUND SIMULATION: On cancellation of a Paid order, a refund record is inserted with status='processed' immediately - simulating an instant refund for the mock system.

6. ORDER STATUS TRANSITIONS: Pending -> Reserved (checkout loaded) -> Paid (payment success) | Reserved -> Failed (payment failed) | Reserved -> Expired (5-min timer) | Paid/Reserved/Pending -> Cancelled (manual cancel)